

Which linear-attention mixer should we ship?

Run 1 results and independent audit · [run_019fe0f9-1bbd-702c-b141-6d58e128bda6](#) · 6 operators × 3 random seeds = 18 training runs · 390M-parameter model, 599,785,472 tokens each · 8×A100-40GB · measured 2026-08-08, audited 2026-08-08 · **Revision 2**

The short version

Ship KDA as we already have it (KDA_BASE), or the variant with the convolution activation removed (KDA_NOACT). The two are indistinguishable in this data; either is defensible.

The decision rests on speed and memory, not on quality. The quality measurement came back *inconclusive* — the experiment was not sensitive enough to tell these six operators apart on quality, and we knew before running it that this was a likely outcome. Speed and memory, by contrast, separate cleanly and are not in doubt.

Do not ship the Householder operators (KDA_R1, KDA_R2) — they are 22–30% slower for no measured quality gain. But see §7: that cost turns out to be a *fixable implementation problem*, not a property of the idea, and the idea itself is available to us for free through a different route.

1 · WHAT THIS WAS FOR, AND THE VOCABULARY YOU NEED

The decision. We are about to spend real money training a large production model. Inside a transformer, every layer has a component that lets tokens exchange information — call it the **mixer**. Standard attention is one choice; it is accurate but its cost grows with the square of sequence length. **Linear-attention** mixers keep a fixed-size running summary instead, so cost grows linearly and generation is cheap. Several competing designs exist. This experiment was run to pick one.

Terms used throughout:

- **Cross-entropy (CE)** — how surprised the model is by text it has never seen. Lower is better. Measured in **nats**. Our models land around 3.05 nats; the differences we are hunting are in the third decimal place (0.01–0.03 nats).
- **Throughput** — training tokens processed per second. Higher is better. Directly proportional to how much the run costs.
- **Peak / reserved memory** — GPU memory used. *Peak* is what the model actually holds; *reserved* is what it asks the allocator for. Reserved is the number that decides whether a bigger model fits on the card.
- **Seed** — the random number that sets the starting weights and the data order. The same design trained with three different seeds gives three slightly different results. That spread is our measurement noise.
- **"Not resolved"** — the difference between two designs was smaller than our noise, so we cannot say which is better. **This is not the same as saying they are equal.**

How the comparison was kept fair. All six versions are byte-identical except for the mixer placed in **two specific layers (slots 6 and 11) out of sixteen**. Everything else — the six standard-attention layers at positions 2, 5, 8, 10, 12, 14, all the feed-forward blocks, the vocabulary, the learning-rate schedule, the data — is the same. So a difference between two versions is caused by the mixer and nothing else.

Two details that were forced on us, and matter later:

- **The six standard-attention layers could not be removed.** Position information (RoPE) is only attached to real attention layers in our codebase. An arm with fewer attention layers would silently lose its sense of word order, and its worse score would look like a mixer effect when it was actually missing position. A previous project lost a result to exactly this.
- **Bigger mixers were paid for by shrinking the feed-forward layers**, so every version has the same parameter count to within $\pm 0.05\%$. Otherwise a "better" mixer might just be a bigger model. This matters in §6 — it is the source of one of the errors we found.

Each version was trained three times (three seeds), which is the minimum that lets you estimate noise at all. As §5 explains, three turned out not to be enough for quality.

2 · THE SIX VERSIONS, IN PLAIN TERMS

Name and mechanism	Config	What is different, and why we included it	Params vs target	Speed ×ref
KDA with no convolution activation KDA_NOACT	conv_activation=None	The shipped operator with one activation function deleted from its short convolutions. Its job is to be a control , not a candidate: it is what makes the gated-conv result below interpretable. It happens to score best on quality — but see §5 before reading anything into that.	−10,080	0.998
KDA plus a gated convolution KDA_GCONV	gated_conv=True depthwise	Adds LFM2/LIV-style learned gates to the short convolutions — a mechanism we built ourselves and wanted to test. Gates are <i>depthwise</i> (one number per channel) rather than <i>low-rank</i> , because low-rank would have added 2.36M parameters (12× our matching tolerance) and broken the seed pairing.	+2,208	0.979
KDA exactly as we ship it today KDA_BASE ← the reference	fla_chunk_kda	Kimi Delta Attention, the operator already in our codebase, running the optimised third-party kernel from the fla library. Every comparison in this document is measured against this row.	−10,080	1.000
KDA-Householder, R=2, negative eigenvalues KDA_R2	num_householder=2 allow_neg_eigval=True	Applies two "reflection" updates per token instead of one, and widens a gate parameter (β) from the range (0,1) to (0,2) so the update can genuinely reflect rather than only shrink. This is the arm we had the most hope for. It runs a hand-written kernel we wrote ourselves , not the optimised library one.	+6,304	0.705
KDA-Householder, R=1, negative eigenvalues KDA_R1	num_householder=1 allow_neg_eigval=True	The same hand-written operator with only <i>one</i> update per token. Included as a control to separate "does a second update help?" from "is our kernel slow?". It answered that question decisively — see §7.	−10,080	0.779
Gated DeltaNet-2 GDN2 Tom Liu's implementation	expand_v=1.0 allow_neg_eigval=False	A different published design that uses <i>two separate</i> gates for erasing and writing to its memory, where KDA drives both from one number. Configured as the paper's headline model. It is the best-tested implementation in our codebase.	+22,688	0.994

Reading the parameter column: the target is 390,135,552 parameters and the tolerance is ±195,068 (0.05%). Every arm is inside it — the largest deviation, GDN2's +22,688, is about 8.6× inside the limit. Parameter matching is not a concern for any result here.

3 · RESULTS — QUALITY, SPEED, MEMORY

Version	Quality (CE, nats)	Spread across seeds	vs reference	Throughput (tokens/s)	×ref	Peak mem GiB	Reserved mem GiB	Plain reading
KDA, no conv activation KDA_NOACT	3.0400	0.00258	−0.0113	418,364	0.998	9.153	10.721	Best point estimate on quality and the most reproducible of all six. Speed and memory tied with the reference. A co-winner.
KDA + gated convolution KDA_GCONV	3.0451	0.01678	−0.0062	410,690	0.979	9.434	11.018	Contains both the single best and nearly the worst individual run in the study. Costs 2.1% speed and 3.1% memory. Verdict: unresolved — see §4.
KDA as shipped (reference) KDA_BASE	3.0514	0.01075	—	419,288	1.000	9.153	10.721	Fastest measured, joint-lowest memory, zero integration risk. A co-winner — and the incumbent.
KDA-Householder R=2 KDA_R2	3.0581	0.00642	+0.0067	295,638	0.705	9.361	13.391	Slowest in the run and +24.9% reserved memory, with no quality gain to show for it <i>at this training budget</i> . Second-most reproducible.
KDA-Householder R=1 KDA_R1	3.0749	0.04516	+0.0236	326,513	0.779	9.216	13.260	By far the least reproducible — its seed-to-seed spread is 17.5× the best arm's . One of its three runs is a clear outlier. See §7 and §8.
Gated DeltaNet-2 GDN2	3.0884	0.00404	+0.0371	416,894	0.994	9.333	10.893	Largest quality gap in the run, and worse in all three seeds. Essentially free on speed and memory. Read §6 before acting on this — the original claim that this was statistically significant was wrong.

A column has been deliberately removed from this revision. The previous version of this document reported **MFU** (model FLOP utilisation — how much of the GPU's theoretical arithmetic capacity was used). That column was wrong, and correcting it makes it redundant: once the underlying bug is fixed, the MFU ranking becomes *identical* to the throughput ranking, so it adds no information. It has been struck rather than repaired. Details in §6.

4 · THE GATED CONVOLUTION: WHY THE ANSWER DEPENDS ON WHAT YOU COMPARE IT TO

This is the most instructive result in the study, and it is a lesson about experimental design rather than about the mechanism.

Compared against **the shipped reference**, the gated convolution looks **better by 0.0062 nats**. Compared against **the no-activation control**, the same arm looks **worse by 0.0051 nats**. **Same arm, opposite sign.**

The reason is exact algebra. The gate our mechanism applies, $2 \cdot \text{sigmoid}(a \cdot u) \cdot u$, is *identically equal* to $(2/a) \cdot \text{silu}(a \cdot u)$ — a SiLU activation with a learnable slope. (Verified to 8.9×10^{-16} , i.e. floating-point exact.) So switching the gate on does not just add a gate: it also **relocates and reparameterises an activation function** that was already there. Comparing against the reference moves three things at once, and no part of that difference can be attributed to any one of them.

The honest comparison is against the control, where the only remaining difference is the gate itself: **+0.0051 nats, 95% CI [−0.031, +0.041], p = 0.77**. That interval spans the entire range of effects we care about, in both directions. The correct statement is "*this experiment cannot tell the gate apart from no gate*" — not "the gate is worse," and not "the gate is fine." Without the control arm, this study would have produced a plausible, publishable, wrong number.

5 · WHY QUALITY CAME BACK INCONCLUSIVE, AND WHY THAT IS THE HONEST ANSWER

Across the 18 runs, the seed-to-seed noise in CE is $\hat{\sigma} = 0.0204$ nats. With three seeds per arm and six arms compared properly, the smallest difference this design could reliably have detected is **0.0636 nats**.

The differences we are looking for, from published work on these operator families, are **0.010 to 0.030 nats**. That is **2–6× smaller than what we could see**. The experiment was outmatched by the question.

The formal results all agree: overall test $F(5,12) = 2.49$, $p = 0.091$ — not significant. No individual comparison against the reference excludes zero once corrected for making five comparisons at once.

Please do not read the quality ranking in §3 as a ranking. The arms are listed in CE order because the table has to be in some order. The gap between best and worst (0.0484 nats) is almost exactly the width of a single confidence interval (0.0484 nats). Any of the six could be genuinely best.

This outcome was written down in advance. The pre-registration filed before the run states that the design might not resolve CE differences and that the recommendation would then fall back to speed and memory. So this is a planned branch, not a failure — and the fallback axes did separate cleanly.

Two things make the quality numbers upper bounds, not estimates.

- **The training budget is small.** Each model saw 1.5 tokens per parameter. Published work clusters at 3–20, and our own production model sits at 27–44. Architecture differences measured at low budgets systematically *overstate*: in our own earlier measurements, one such gap halved ($0.0103 \rightarrow 0.0059$ nats) when the budget grew 15×. So every CE difference here should be read as a ceiling on what it would be at production scale.
- **A null result is a bound, not a proof of equality.** We did not pre-declare an equivalence margin, so we are not licensed to claim any two arms are the same — only that any difference is smaller than roughly 0.048 nats.

The useful by-product: $\hat{\sigma} = 0.0204$ nats is the first real measurement of noise at this scale in this project. It replaces a 5.5×-wide guess with a factor-2.3 bracket, and any future experiment should be sized from it.

6 · WHAT THE INDEPENDENT AUDIT CHANGED — READ THIS BEFORE QUOTING REVISION 1

After the results were written up, twelve independent reviews re-derived every number from the raw per-run data, the analysis code, the model source, and the AWS logs. **The core statistics all reproduced exactly** ($\hat{\sigma}$, the overall test, the correction factor, the detection limit — all confirmed to five significant figures, and cross-checked against `scipy`). **The primary quality numbers were also confirmed bit-for-bit against the original training logs.**

But six specific claims in revision 1 were wrong. All six were in the *hand-written* summary, not in the automatically generated analysis, which was clean throughout. They are corrected here.

Revision 1 said	The truth	Why it happened, and does the decision change?
"GDN2 is significantly worse — the only arm significant under any test," CI [+0.0262, +0.0479], excluding zero.	No comparison in this study is statistically significant. The correct interval is [−0.0008, +0.0749] , which includes zero . Our own analysis script outputs exactly that, with <code>excludes_zero: false</code> — on all five comparisons.	The published interval implies a statistical cut-off of 1.64 , which is smaller than the 1.96 used for a <i>single uncorrected</i> comparison. A correction for five comparisons can only make an interval wider. The centre was right; the width was not. Decision: unchanged — but GDN2 must be described as "largest point estimate, not resolved," never as "significantly worse." This matters for how we talk to Tom Liu about it.
"Under the pre-registered unequal variance fallback..."	That fallback was never triggered. The pre-registration names one test (Levene) as the decider, and says to switch methods only <i>if it rejects</i> . It did not ($p = 0.48$). A second test (Bartlett) did reject, but was explicitly designated "reported, not deciding."	Switching to the method that produces a significant answer, after seeing that it does, is precisely what a pre-registration exists to prevent — and ours said so in those words. The generated report got this right; the summary drifted. Decision: unchanged.
"Kernel overhead is 28.6%; the second reflection costs a further 10.9%."	The real figures across all three seeds are 28.4% and 10.4% .	The published pair came from a single run (one seed, median step time) quoted in the same sentence as three-seed averages — two different measurements presented as one. Decision: unchanged; the attribution logic in §7 is unaffected.
"KDA_R1 is the same function as the reference, so its slowdown is pure kernel overhead."	It is parameter- and FLOP-identical, but it is not the same function : it also widens β from (0,1) to (0,2), computes its gate differently, and calls a different kernel. There is no arm in this study that isolates the reflection count alone.	Consequence split: the speed and memory conclusion survives (widening β is one multiplication and changes FLOPs not at all), but the quality and reproducibility conclusions about $R=1$ do not — they confound two changes. Decision: unchanged, wording tightened.
"GDN2 has the highest MFU in the run (42.0%)."	An artifact. GDN2 is in fact slower than the reference (0.994×). Corrected to one convention, the reference has the highest MFU and the MFU order becomes identical to the throughput order.	Our FLOP counter mixes two conventions: feed-forward blocks and GDN2 count forward+backward work, while KDA and Householder count forward only — a systematic 3× advantage on GDN2's mixer term. Also, because bigger mixers were paid for by shrinking feed-forward layers (§1), an arm with a <i>larger</i> operator can report <i>fewer</i> total FLOPs. Decision: unchanged — throughput is measured by a stopwatch and never touches the FLOP count. MFU column struck.
"A well-powered null result."	Badly under-powered , and correctly reported as a bound. Detection limit 0.0636 against a 0.010–0.030 target.	A wording error with the opposite meaning to the truth. Decision: unchanged.

Three housekeeping defects, none of which move a conclusion

- **The machine-readable `results.json` in the results folder is a stale snapshot** taken mid-run. It describes 8 of 18 runs, lists three arms as "NO DATA," and reports the gated-conv comparison with the *opposite sign*. Anyone parsing it instead of reading this document gets a materially wrong answer. **It must be regenerated before this branch is merged.**
- **16 of the 18 per-run result files were transcribed by hand**, not captured programmatically. Detected by an exact arithmetic identity that must hold in binary (per-device throughput × 8 = total) and fails in 13 of 18 files. **The important fields are unaffected** — quality and both memory figures are full precision in all 18 and match the original logs exactly; the rounding on throughput is at most 0.026%, against arm differences of 2–29%. But the files should be re-extracted from the logs.
- **A guard was silently disabled by its own configuration.** The analysis script keeps a list of "fields we have accounted for" so it can warn about fields nobody checked. Five fields — including the broken MFU and FLOP figures — were on that list but never actually read anywhere in the 3,200-line script. The warning therefore came back empty, and the generated report skips from section 5 to section 7 because section 6 had nothing to print. **The check built to catch exactly this bug was switched off by declaring the bug's fields as checked.**

7 · THE MOST USEFUL FINDING: THE REFLECTION MECHANISM IS AVAILABLE TO US FOR FREE

The 22% slowdown we measured is not the price of the mechanism. It is the price of our own unfinished kernel — and we do not need that kernel.

"Negative eigenvalues" sounds like a deep change to the mathematics. In implementation it is **one multiplication**: a gate value β computed as $\text{sigmoid}(x)$ becomes $2 \cdot \text{sigmoid}(x)$. That is the whole mechanism. It is a change to how one number is *produced*, not to the kernel that consumes it.

Concretely, in our codebase and in the third-party library we already depend on:

- The optimised library kernel `chunk_kda` **already accepts this option** as a documented public argument, implemented as a single scale factor. **No GPU kernel in the library references it at all** — the fast code is simply unaware of β 's range.
- There is **no range check, no clamp, and no upper bound** on β anywhere in that code path. The matrix the algorithm inverts has ones on its diagonal and zeros above, so its determinant is exactly 1 *for any value of β* — the widening cannot make it singular.
- Our shipped `KimiDeltaAttention` **already contains the two lines**, behind a config flag it already exposes. Our `GatedDeltaNet` has been training with the wide range on the fast kernel **by default, for the entire project**.
- The paper that introduced the idea states it outright: "*for DeltaNet, implementing the extended eigenvalue range is straightforward, since there is no need to modify the Triton kernel.*"
- Stability was **already measured in our own tree**: the wide- β regime reached $\beta = 1.93$ and landed in the *same* numerical-accuracy band as the baseline, with all test cells finite.

So the fix is one configuration field, not an engineering project. Our hand-written Householder kernel, whose own documentation says it is a sequential implementation "expected to be materially slower" and built for "mechanism validation, not throughput," was never needed to get this mechanism at $R=1$.

Why our kernel is slow — diagnosed

It walks the sequence one position at a time: **4,096 dependent iterations**, three times per training step. It uses **no tensor cores** at all, and runs at about **3.7% occupancy** — achieving **0.22% of the GPU's arithmetic peak**. It is limited by latency and serialisation, not by the amount of arithmetic. The optimised library kernel instead processes 64 positions at a time.

The extra reserved memory has a single identified cause: the backward pass keeps a full-precision copy of the running state for every position, ~2 GiB at our shape. Good news — it is freed per layer, so it does **not** grow with the number of layers using the operator. It does grow with batch size and sequence length.

But be careful what you claim it buys

In this run, the wide- β arm scored **+0.0236 nats** — **worse** than the reference, and well inside the noise. Two published sources also report no consistent language-modelling gain from this change.

What it demonstrably buys is **state tracking** — the ability to represent algorithmic patterns a contracting update cannot. In the source paper, a parity task goes from **0.233 to 0.982** accuracy with this one change.

So the correct pitch is free optionality, not free quality. Turning it on costs nothing measurable, and makes a capability available. It is not a quality upgrade, and should not be sold as one.

Note this only covers $R=1$. Getting *two or more* reflections per token at full speed still needs kernel work — see the box below.

If we later want more than one reflection per token ($R > 1$), start with a measurement

A plan was on record to make the $R>1$ case fast by re-expressing it on top of an existing, already-optimised library kernel (`chunk_dplr_delta_rule`). An independent review of that plan found the idea sound but three of its details wrong, and it changes what an engineer should do first:

- **The risk we were most worried about is not real.** We feared the library's backward pass would be numerically unstable under the padding trick the plan needs. There are **zero divisions** anywhere in that kernel's source, and the padding pattern is *already what the library itself ships* for this exact case. A different, milder risk was found instead: one intermediate matrix becomes ill-conditioned precisely when β is large — worth watching, not blocking.
- **The memory saving was overstated.** The claim "1.00 GiB $\rightarrow$ 32 MiB" is the $R=1$ number. At $R=2$ it is closer to **64–176 MiB**. Still a real 6–17 $\times$ improvement, so the memory argument survives — but do not quote 32 MiB. (Our own kernel's source also overstates its own workspace by 1.5 $\times$, on the numbers used to size this decision.)
- **It might not actually be faster.** The replacement kernel builds four internal matrices where the simpler one builds one, works in 16-position chunks rather than 64, and recomputes its forward pass during the backward. **It could land slower than the 0.705 $\times$ it is meant to fix.**

Therefore: day one is a half-day throughput benchmark, not a week of implementation. That single measurement either justifies the work or saves it. And do *not* modify the other candidate kernel (`chunk_gated_delta_product`) — its backward pass is shared with the production `GatedDeltaNet` path, so patching it would put our best-throughput arm at risk.

8 · A WARNING ABOUT REUSING THE SPEED NUMBERS

The treatment operator ran in **only 2 of 16 layers**. Everything else was identical, so the entire slowdown came from those two layers. Divide it out and the operator itself is running **roughly 5× to 21× slower** than the shipped one — not 22% slower. The whole-model figure is diluted by the fact that most of the model was untouched.

This means the penalty grows if we use the operator in more layers, which is the usual direction for these architectures:

Layers using the Householder operator	2 of 16 (measured)	4 of 16	8 of 16	14 of 16
Whole-model throughput vs reference, R=1	0.779×	0.638×	0.468×	0.335×
Whole-model throughput vs reference, R=2	0.705×	0.536×	0.366×	0.248×

Do not quote 0.779× or 0.705× for any design that uses a different number of linear-attention layers. The model reproduces the measured value exactly at 2-of-16, which is what makes the extrapolation trustworthy. Memory does not scale this way and is not the blocker — time is.

9 · SHOULD WE SPEND MORE COMPUTE ON THIS? — COSTED

This is the question the results provoke, so it was priced properly. Everything is at \$21.96 per node-hour, and includes the held-out evaluation pass (the previous revision's per-run cost omitted it and was ~1.8× too low).

What resolving quality would cost

Goal	Seeds per arm	Runs	Cost
Where we are now	3	18	spent
Detect a 0.030-nat difference	11	66	~\$2,900
Detect a 0.010-nat difference	95	570	~\$6,300–11,300

What being wrong would cost

If we pick the worse operator and its true quality deficit is 0.010–0.030 nats, the fix is simply to train on a few percent more tokens. On a 1B-parameter production run (27–44B tokens, roughly \$1,000–1,650) that is:

True quality deficit	Extra tokens needed	Extra cost
0.010 nats	+7.6%	\$77
0.020 nats	+15.9%	\$161
0.030 nats	+25.1%	\$253–484

The arithmetic that settles it.

The entire quality ambiguity is worth **\$77–484** on the production run. Removing it costs **\$2,900–11,300**. Meanwhile the *speed* choice — which is already resolved to about 1% — carries a **larger** production penalty (**\$286–687**) than the quality question does.

So: do not buy more seeds. Do not buy more tokens. Training longer is plausibly worse on both axes at once, because these architecture differences *shrink* with budget faster than the noise does, while cost rises in proportion.

What is worth buying

- 1. **The one-field experiment, ~\$55–65.** Three runs on the same frozen seeds, with the reflection range turned on via the *fast* kernel. This gets the mechanism at full speed and tests whether the reproducibility problem was our kernel. **Gate it** on two receipts: a logged β above 1.0 (proving the flag took effect) and throughput parity with the reference.
- 2. **A recall-focused re-scoring, ~\$79–171 plus 2–4 engineer-days.** All 18 final checkpoints are still in S3, so no retraining is needed. Aggregate quality averages over mostly-easy tokens; the published evidence is that these operators differ on *recall*, where one study found a 20-point gap at near-identical perplexity. This is the only option that returns a new *kind* of evidence. **Not free:** the mask-builder script does not exist yet, and the scoring path is single-rank and will hang on 8 GPUs until fixed.
- 3. **Optional, ~\$220–250:** a focused two-arm reference-vs-GDN2 comparison, if that result needs formally closing rather than just correctly describing.

10 · WHAT WE DID NOT MEASURE — THE HONEST GAPS

- **Nothing about inference.** We measured training throughput only. The entire practical argument for linear attention is cheap single-token generation, and that path was never timed. For a production decision this is the largest gap.
- **Only one configuration.** One sequence length (4,096), one batch shape, one node, one GPU type. The four fast arms are within 2.1% of each other — close enough that a different shape could re-order them.
- **No recall or long-context measurement.** The secondary endpoint designed for exactly this was never wired up (see §9, item 2).
- **No test in our codebase has ever checked kernel accuracy at β above 1** — which is the regime both Householder arms actually ran in. The existing β test recomputes the arithmetic inside the test itself and never calls the kernel. This is a real gap in the shipped result, not only in the proposal.
- **The one test that checks our fused kernel does not read future tokens requires a GPU**, and we found no evidence it was run before these 18 jobs launched. On a machine without a GPU it skips silently, which looks identical to passing.
- **The three "data seeds" are not three orderings of the same data.** Each run sees only 0.24% of the corpus, so they are ~99.8% different samples. Harmless for comparing arms (all six share a seed's data), but it means our noise figure mixes optimisation noise with data-sampling noise.

Revision 2, 2026-08-08. Supersedes revision 1; six corrections in §6, MFU column struck, costs re-priced to include evaluation. Verified against the 18 raw per-run files, the analysis script's own output, the model source at commit `9425fea`, `fla-core-0.5.1` source, and CloudWatch logs. Primary quality figures confirmed bit-for-bit against the original logs. Statistics: pooled $\hat{\sigma} = 0.020415$ nats at 12 degrees of freedom, χ^2 interval [0.01464, 0.03370]; $F(5,12) = 2.4893$, $p = 0.0908$; Dunnett critical value 2.9013 at $k = 5$; detection limit 0.06360 nats — all independently reproduced and cross-checked against `scipy.stats.dunnett`. Full working: `docs/mixer-bakeoff/run1/REPORT.md` (generated, clean) and `docs/mixer-bakeoff/HANDOFF.md`. Branch `edullm/mixer-bakeoff`.